

KNr.

MNr.

Zuname, Vorname

Studium:

☐

Informatik (Bakk.)

☐

Elektrotechnik (Master)

Ges.)(100)

1.)(30)

2.)(30)

3.)(40)

Zusatzblätter:

Bitte verwenden Sie nur dokumentenechtes Schreibmaterial!

1 Synchronisation mit Semaphoren (30)

Ein Softwarepaket aus vier Prozessen soll eine Anlage simulieren, die Kartons mit Würfeln und Murmeln befüllt. Zwei gleichartige Prozesse simulieren den An- und Abtransport von Kartons zur bzw. von der Befüllanlage. Je ein Prozess kontrolliert das Einfüllen von Murmeln bzw. Würfeln in einen bereitstehenden Karton.

Ergänzen Sie die gegebenen Codestücke mit *Semaphoroperationen*, um die Prozesse entsprechend der unten gegebenen Bedingungen zu synchronisieren. Das Verwenden von globalen Variablen zur Synchronisation oder Kommunikation zwischen Prozessen ist nicht erlaubt.

- Die beiden Prozesse für den An- und Abtransport der Kartons sollen Roboter simulieren. Jeder Roboter hat einen eigenen Vorrat an leeren Kartons und einen eigenen Ablageplatz für volle Kartons und führt wiederholt folgende Aktionen durch:
 - Karton vom Vorrat holen: *karton_aufnehmen()*.
 - Karton zur Befüllanlage transportieren und öffnen: *karton_bereitstellen()*.
 - Karton verschließen und von der Befüllanlage nehmen: *karton_entfernen()*.
 - Karton zum Ablageplatz abtransportieren: *karton_abtransportieren()*.
- Der Prozess Murmelspender ruft in einer Endlosschleife die Funktion *Murmel_liefern()* auf. Jeder Aufruf dieser Funktion lässt eine Murmel in den Karton auf der Befüllanlage fallen.
- Der Prozess Würfelspender ruft in einer Endlosschleife die Funktion *Wuerfel_liefern()* auf. Jeder Aufruf dieser Funktion lässt einen Würfel in den Karton auf der Befüllanlage fallen.
- Die Prozesse sind so zu synchronisieren, dass die Parallelität ihrer Aktionen die folgenden Regeln erfüllt, darüberhinaus aber möglichst wenig eingeschränkt wird.
- Auf dem Befüllplatz der Anlage ist Platz für einen Karton. Synchronisieren Sie die Prozesse so, dass die Roboter die Befüllanlage abwechselnd mit Kartons versorgen.
- Jeder Karton ist mit N Murmeln und K Würfeln zu befüllen. Stellen Sie die korrekte Befüllung der Kartons durch die Synchronisation sicher.

(a) Initialisierungen (vor Start der Prozesse)

```
init(K, 1);  
init(W1, 0);  
init(W2, 0);  
init(M1, 0);  
init(M2, 0);
```

In der Folge sind Codestücke für die zu synchronisierenden Prozesse gegeben, die bereits alle notwendigen Funktionsaufrufe enthalten. Ergänzen Sie in diesen Codestücken die fehlenden Semaphoroperationen zur Synchronisation.

(b) Prozesse für die Befüllung

```
/** Murmelspender **/
```

```
do {
```

```
    wait(M1);
```

```
    Murmel_liefern();
```

```
    signal(M2);
```

```
} while(1);
```

```
/** Wuerfelspender **/
```

```
do {
```

```
    wait(W1);
```

```
    Wuerfel_liefern();
```

```
    signal(W2);
```

```
} while(1);
```

(c) Prozesse für Roboter

```
/** Roboter R1 **/
```

```
do {
```

```
    karton_aufnehmen(R1);
```

```
    wait(K);
```

```
    karton_bereitstellen(R1);
```

```
    for (int i=0; i<N; i++) {signal(W1);}
    for (int i=0; i<K; i++) {signal(M1);}
```

```
    for (int i=0; i<N; i++) {wait(W2);}
    for (int i=0; i<K; i++) {wait(M2);}
```

```
    karton_entfernen(R1);
```

```
    signal(K);
```

```
    karton_abtransportieren(R1);
```

```
} while (1)
```

```
/** Roboter R2 **/
```

```
do {
```

```
    karton_aufnehmen(R2);
```

```
    wait(K);
```

```
    karton_bereitstellen(R2);
```

```
    for (int i=0; i<N; i++) {signal(W1);}
    for (int i=0; i<K; i++) {signal(M1);}
```

```
    for (int i=0; i<N; i++) {wait(W2);}
    for (int i=0; i<K; i++) {wait(M2);}
```

```
    karton_entfernen(R2);
```

```
    signal(K);
```

```
    karton_abtransportieren(R2);
```

```
} while(1);
```

2 Page Replacement (30)

Gegeben ist ein Arbeitsspeicher mit vier Frames, dessen Seiten mit unterschiedlichen Ersetzungsstrategien ersetzt werden sollen: mit OPT, LRU, bzw. mit dem Clock Algorithmus. Ergänzen Sie in den Tabellen für jeden Algorithmus die Speicherinhalte für jeden Frame nach jedem Zugriff der angegebenen Seitenzugriffsfolge an. Die Seitenzugriffsfolge ist für alle Algorithmen gleich. Sie ist jeweils in der Kopfzeile der Tabelle gegeben. Geben Sie in den Spalten die Speicherbelegung nach dem entsprechenden Seitenzugriff an und kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Der Arbeitsspeicher ist am Beginn leer. Für die ersten vier Speicherzugriffe sind die Tabellen bereits ausgefüllt.

OPT-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	E	E	E	E	E	E	E	E	E	E	E	E
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X				X							

LRU-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	F	F	F	F	F	F	F	F
1		B	B	B	B	B	B	B	B	B	B	D	D	D	D	D	D
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	E	E	E	E	E	E	E
PF	X	X	X	X		X			X	X	X	X					

Clock-Algorithmus:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	E	E	E	E	E	E	E	E
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X			X	X							

Vergleichen und diskutieren Sie die Anzahl der bei den Seitenersetzungsstrategien beobachteten Page Faults. Entspricht das Ergebnis dem zu erwartenden Verhalten?